

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Фронт-энд разработка

Отчет

Лабораторная работа 2

Выполнил:

Смирнов Александр

Группа J3210

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г

Задача

В рамках второй лабораторной работы необходимо было интегрировать клиентскую часть платформы с сервером с использованием json-server.

Реализованный функционал:

- Настройка взаимодействия с API с помощью асинхронных запросов (Fetch API, async/await).
- Замена локальных мок-данных на получение списка моделей и датасетов с сервера (GET).
- Реализация авторизации и регистрации с проверкой и записью пользователей в базу данных сервера (GET, POST).
- Добавление новых моделей/датасетов через форму с отправкой данных на сервер (POST), включая новые поля спецификации (Metrics, Usage).
- Взаимодействие с карточками: обновление счетчика звезд и скачиваний на сервере (PATCH).
- Реализация системы вложенных комментариев с сохранением на сервере.
- Клиентская генерация файла при нажатии на кнопку скачивания модели.

Ход работы

1. Интеграция API для загрузки и фильтрации данных

Локальные массивы данных были заменены на асинхронные запросы к `http://localhost:3000`. При загрузке главной страницы отправляется GET-запрос к эндпоинту `/items`. Фильтрация и поиск теперь работают на основе данных, полученных с сервера, после чего происходит динамический рендер карточек. Для демонстрации, на рис. 1 изображена главная страница с открытой консолью разработчика, где видно успешный GET-запрос к `/items`.

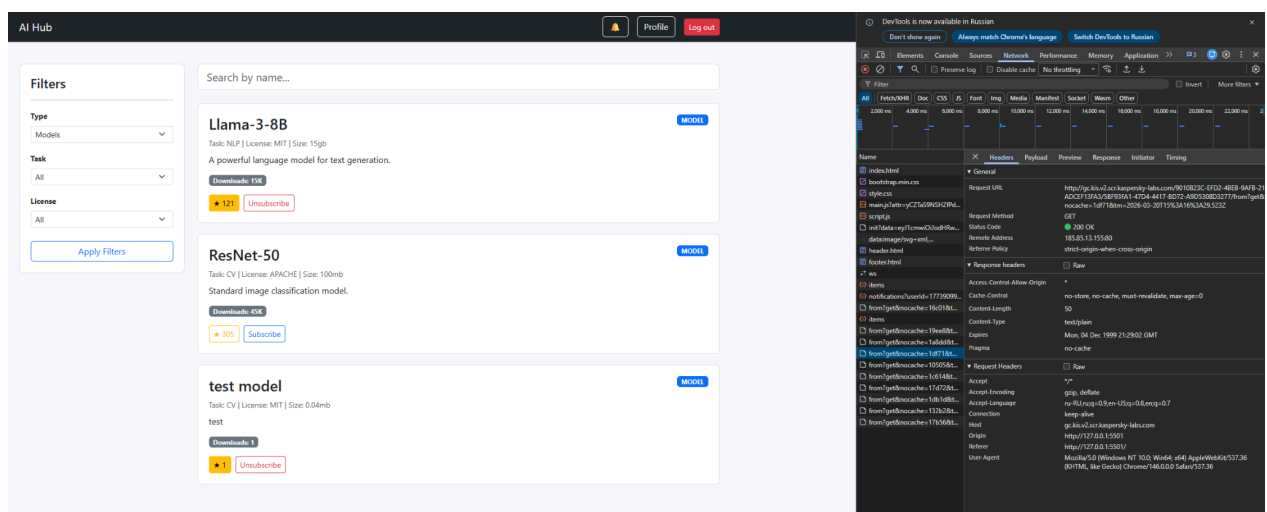


Рис. 1. Демонстрация отправки запроса через вкладку Network

2. Авторизация и регистрация через API

Логика авторизации была переписана. При входе форма отправляет GET-запрос с параметрами email и password. Если сервер возвращает пользователя, сессия сохраняется в localStorage. При регистрации сначала проверяется уникальность email, после чего отправляется POST-запрос для создания нового пользователя в db.json. На рис. 2 пример того, как в db.json выглядят зарегистрированные пользователи.

```
},
"users": [
  {
    "id": "1773909911123",
    "username": "test alex",
    "email": "a@a",
    "password": "123"
  },
  {
    "id": "1773995209973",
    "username": "test2",
    "email": "b@b",
    "password": "123"
  }
],
```

Рис 2. Скриншот из файла db.json, где видно созданных пользователей

3. Публикация новых моделей и датасетов

В форму загрузки (рис. 3) в личном кабинете были добавлены новые поля: Metrics и Usage Example. При сабмите формы собирается объект с данными и отправляется POST-запрос на сервер. Новая карточка сразу же становится доступна в каталоге и отображается во вкладке My Uploads.

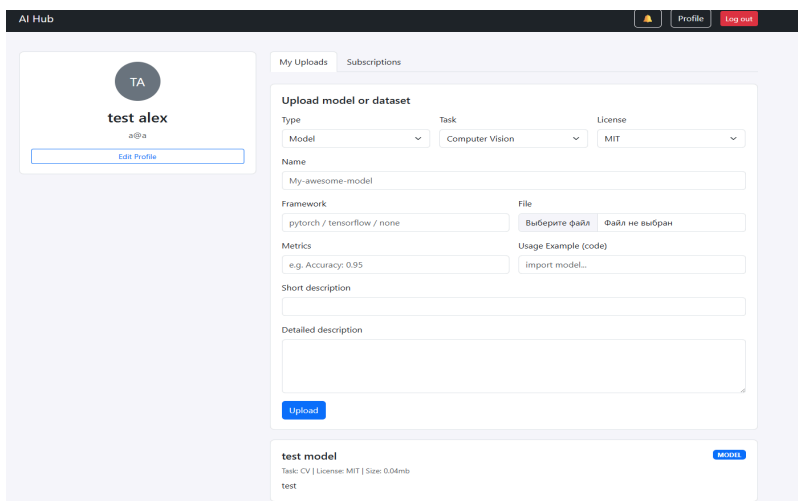


Рис 3. Обновленная форма добавления карточки

4. Система обсуждений

На странице детального просмотра модели/датасета реализована система комментариев (рис. 4). Комментарии подгружаются GET-запросом с фильтрацией по itemId. Реализована поддержка вложенности через атрибут parentId. Добавление нового комментария вызывает POST-запрос, после чего список комментариев обновляется динамически без перезагрузки страницы.

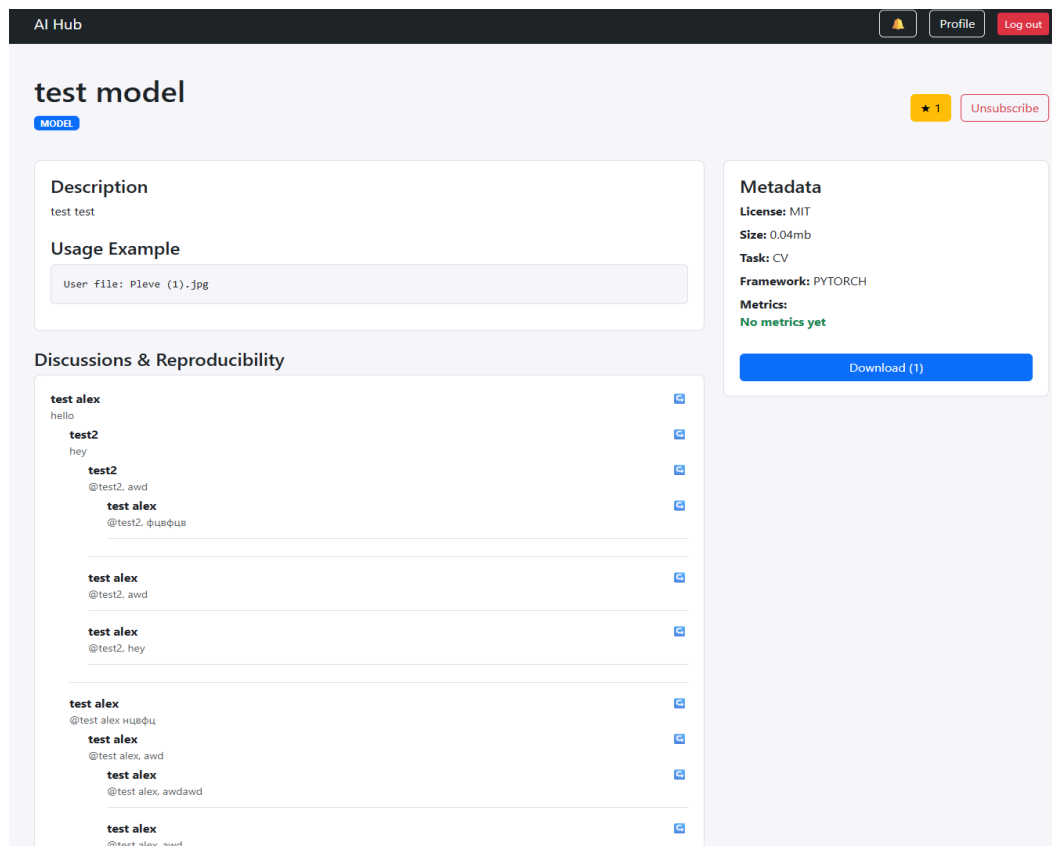


Рис. 4. Пример обсуждения под карточкой

5. Интерактивные действия: Лайки и Скачивания (PATCH-запросы и Blob)

Реализовано точечное обновление данных на сервере без перезаписи всего объекта:

- Звезды: При клике на кнопку Star счетчик изменяется локально, после чего на сервер уходит PATCH-запрос с обновленным количеством звезд.
- Скачивание: При нажатии на кнопку скачивания скрипт парсит текущее количество загрузок (например, переводит 15K в 15000), прибавляет 1 и отправляет PATCH-запрос. Одновременно с этим на стороне клиента с помощью API Blob генерируется .txt файл (рис. 5), содержащий

метаданные модели, и автоматически начинается его скачивание браузером.

```
AI Hub Download
Name: test dataset
Task: cv
Framework: pytorch
License: mit

Metrics: 100% usefull
Usage:
import datasets; data = datasets.load("test")
|
```

Рис. 5. Скачанный .txt файл

Вывод

В результате выполнения лабораторной работы проект был успешно переведен на архитектуру клиент-серверного взаимодействия.

Были получены и закреплены следующие навыки:

- Работа с Fetch API и асинхронным кодом в JavaScript (Promises, async/await).
- Понимание семантики HTTP-методов и их применение в REST API (GET для получения, POST для создания, PATCH для частичного обновления).
- Синхронизация состояния пользовательского интерфейса с серверной базой данных.
- Использование Web API (в частности, объекта Blob и URL.createObjectURL) для динамической генерации и скачивания файлов на стороне клиента без участия бэкенда.
- Обработка ошибок сети и валидация данных перед отправкой на сервер.

Проект стал полноценным динамическим веб-приложением, готовым к подключению реального бэкенда.